The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

9

The Shopping Basket, and Where State Lives

Part III — From Basket to Order

Taught in	Week 7
Laboratory	Session, cookie and database baskets across a restart
Assessment	—

This is a class extract. It is *Chapter 9* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 9

The Shopping Basket, and Where State Lives

Part 1 | Part 2 | **Part 3** | Part 4 | Part 5 | Part 6

Part III — From Basket to Order

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why HTTP has no memory, and **name** the three places a shop can keep what it remembers about a visitor.
- **Locate** a basket in a running shop, and **state** which store holds it, with evidence rather than with belief.
- **Measure** what a session costs in rows, bytes and queries, and **distinguish** the cost of a visitor from the cost of a customer.
- **Predict** what survives a restart, a redeployment and a volume deletion, and **verify** each prediction.
- **Judge** a session configuration — engine, expiry, cookie flags — against what the shop actually does.

Before You Start

Chapter 5: containers, volumes, and the difference between restarting a container and deleting one. Chapter 6: the tables behind the screens, and how to read a query the server actually ran rather than one you assumed it ran. From Web Technologies: HTTP requests and responses, and what a cookie is.

Key Terms in This Chapter

state • stateless protocol • cookie • session • session identifier • session engine • server-side state • client-side state • persistent cart • abandoned basket • session expiry • garbage collection • volume • cache miss

9.1 The Problem: A Protocol With No Memory

A customer puts a laptop in the basket, browses to another category, comes back, and the laptop is still there. Nothing about HTTP makes that happen.

Every HTTP request is independent. The server receives a request, answers it, and keeps nothing. The next request from the same person arrives looking exactly like a request from a stranger. If the shop is to know that these two requests belong to the same visitor, *something carried between them* must say so, and that something can only travel in the request itself — because the request is all there is.

State, Cookie, Session

State is anything the shop remembers between one request and the next: what is in the basket, who is logged in, which currency is being shown.

A **cookie** is a small named value the server asks the browser to store and send back on every subsequent request to that site. It is the only standard way to make a second request

identifiably related to a first. It lives on the customer's machine, which means the customer can read it, change it and delete it.

A **session** is state the *server* keeps, filed under a key. The browser carries only the key — the **session identifier** — in a cookie; the data stays on the server, where the customer cannot touch it.

The distinction is the whole of this chapter. A cookie is a place. A session is an arrangement that *uses* a cookie to point at a place somewhere else.

9.2 Three Places State Can Live

Given that the browser must carry something, a shop has three choices about *how much* it carries, and they trade off against each other in ways worth seeing before measuring anything.

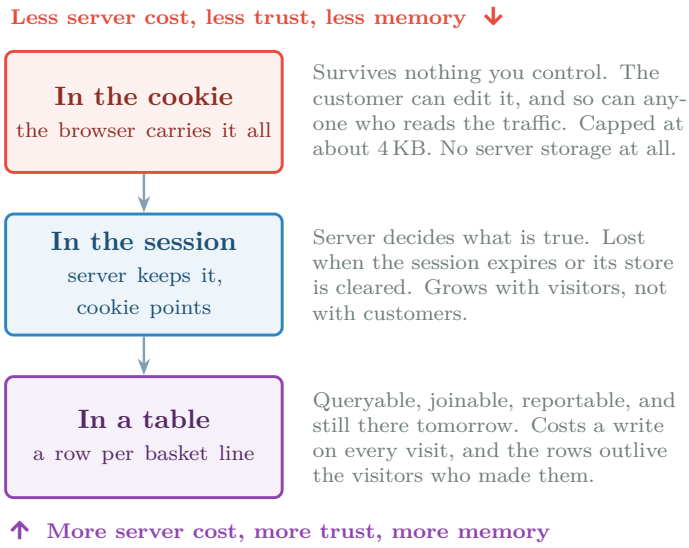


Figure 9.1: The three stores, and what each one buys. Nothing forces a shop to pick one: most use all three at once, for different things. The question worth asking of any shop is not which one it uses, but which one holds the basket.

A rule of thumb, before any measurement

Put in the **cookie** only what you would not mind the customer editing — a chosen language, a dismissed banner. Put in the **session** what must be true for this visit and need not outlive it. Put in a **table** anything you will later want to count, report on, or still have tomorrow.

Applying that rule to a basket gives an answer immediately: a shop wants to know how many baskets were abandoned last week, so the basket belongs in a table. That is an argument. The rest of this chapter is evidence.

9.3 Where This Shop Actually Puts It

Almost every tutorial on this subject says the cart is kept in the session. Before believing that of the shop in front of you, look.

The method is the one Chapter 6 arrived at the hard way: do not reason about what the code probably does, make the shop do it and read what changed. Add three products, and after each one measure all three stores at once.

Measured: Add three products; watch all three stores

Produced by `code/m09_basket.py -where` against the stack in Appendix A. "Cookie" is what the browser sends up on every request; "session" is the length of the stored session record; "cart" is rows in `oc_cart` and the bytes in them.

After adding	Cookie	Session	Cart rows	Cart bytes
(nothing)	49 B	18 B	0	0
2 × MacBook	49 B	18 B	1	53
1 × iPhone	49 B	18 B	2	106
3 × HTC Touch HD	49 B	18 B	3	159

The cookie does not move, and the session does not move. Both are constant across an empty basket and a three-line one. The entire session record is eighteen bytes:

```
{"currency":"USD"}
```

The basket is a table. It is rows in `oc_cart`, one per line, about 53 bytes each, keyed by the session identifier. The cookie carries a 26-character identifier and a currency, 49 bytes in all, and that is the whole of what the browser knows.

So the common claim is false here. It is worth knowing *why* it is so widely believed: in a simpler shop it is usually true, and the arrangement OpenCart uses — session for identity, table for the basket — is the one a shop adopts when it wants to report on baskets that were never checked out.

"The cart is in the session" — check before you repeat it

This matters beyond trivia. If you believe the basket is in the session, you will also believe:

- that lengthening the session expiry lengthens the life of a basket (here it does both, but for two different reasons, in two places);
- that switching the session engine moves the basket (it does not — see the next measurement);
- that clearing sessions clears baskets (it does not; it orphans them);
- that you cannot report on abandoned baskets without extra work (you can: they are rows, and Chapter 17 reads them).

Each of those is a decision someone will make on a Friday afternoon. The evidence took one command.

9.4 What a Session Costs

The basket is in a table, but the shop still keeps sessions, and sessions are not free. Two questions: how many are created, and what does each one cost per request?

Measured: A session is written before the visitor asks for anything

From `code/m09_basket.py -sessions`. Twenty requests made as twenty first-time visitors, then twenty made by one visitor who keeps the cookie.

20 first-time visitors	**20** new session rows
1 visitor, 20 page views	**1** new session row

The session table on this machine held **645 rows, about 35.6 KB**, accumulated entirely by the measurements in this handout — no human has ever shopped here.

Every request writes a session, including the first. A visitor who loads one page and leaves has cost a row. A crawler that ignores cookies costs one row per URL it touches, and the rows are removed on a clock rather than on departure: `config_session_expire` is **86,400 seconds**, a full day.

This is the first number in the course that scales with *traffic* rather than with *business*. The catalogue in Chapter 6 grew because the shop sold more things. This grows because more robots visited.

Measured: Database sessions against file sessions

From `code/m09_basket.py -engine`. OpenCart ships three session engines — `db`, `file` and `redis` — selected in `system/config/catalog.php`. The storefront ships configured for `db`. Both rows below are the same home page, one engine apart.

Engine	Page	Queries	On session	Cart rows	Files
`db`	89 ms	117	2	1	1
`file`	71 ms	115	0	1	14

Exactly two queries per request are the session's — one `SELECT` to read it and one `REPLACE` to write it back. Moving to files removes both and takes about 20% off the page on this machine, where the database and the web server are two containers on one host.

The basket did not move. The `Cart rows` column is 1 under both engines. The engine decides where eighteen bytes live; it has nothing to do with the basket.

Files are not free either, they are just billed elsewhere. Each session file holds those same eighteen bytes and occupies a 4 KB filesystem block — an amplification of more than two hundred. And a file session is tied to one machine's disk, so the moment the shop runs on two web servers, a customer served by the second one is a stranger. That, not speed, is why the shipped default is `db`.

Reading that 20% honestly

It was measured on one host where the database is a container a virtual network hop away. On a deployment where the database is a separate machine, removing two round trips per request would save more. On a deployment behind a load balancer, the file engine would not work at all.

A measurement tells you what happened on the machine it ran on. Appendix A names that machine for exactly this reason. Quoting 20% as a property of OpenCart would be a misuse of your own result.

9.5 Two Things the Source Does That the Screens Do Not Show

Reading `system/library/cart/cart.php` and `system/library/session.php` turns up two behaviours that no admin screen mentions and both measurements confirm.

Measured: An empty basket costs more than a full one

From `code/m09_basket.py -cache`. The same home page, with the basket holding 0, 1, 2 and 3 lines.

Basket lines	Queries	SELECT on cart	DELETE on cart
0	117	**6**	1
1	119	1	1
2	126	1	1
3	129	1	1

An empty basket is read six times; a full one is read once. The cache in `Cart::getProducts()` begins `if (!$this->data)`, which is true both when the basket has not been loaded yet *and* when it is empty. PHP cannot tell those apart that way, so an empty basket misses its own cache on every call — and a shop's commonest visitor has an empty basket.

Every storefront page deletes before it reads. The `DELETE` column is 1 on every row, including the empty basket. The `Cart` constructor removes expired guest baskets each time it is built, so a page view is a write even when the visitor only looked.

Measured: One request in five rebuilds the session table

From `code/m09_basket.py -gc`, over two runs.

25 requests	`OPTIMIZE` ran 7 (28%)	555 ms in total
60 requests	`OPTIMIZE` ran 15 (25%)	889 ms in total

`Session::__construct` registers `gc()` as a shutdown function, so garbage collection is *considered* on every request; `session_probability` and `session_divisor` make it *run* on about one in five. When it runs it deletes expired rows and then issues `OPTIMIZE TABLE`, which on InnoDB is a full table rebuild — around 60 ms here, on a 645-row table.

Both runs are printed because neither is the answer. The configured rate is one in five; 7 of 25 and 15 of 60 are what one in five looks like in small samples, and reporting only the first would have claimed 28%.

9.6 What Survives

Chapter 5 introduced volumes with the claim that they are what makes a container's data outlive the container. A basket is the smallest honest test of that claim, because you can put one somewhere and then try to destroy it three different ways.

Measured: Three ways to stop a shop, and what is left afterwards

From `code/m09_basket.py -restart -destroy`. A three-line basket is created, then each operation is applied in turn and the basket looked for again — in the tables, and in the page the same browser is shown.

Operation	Took	Cart rows	Session rows	Basket shown
— (before)	—	3	1	yes
`restart shop`	3 s	3	1	yes
`down`, `up -d`	10 s	3	1	yes
`down -v`, `up -d`	17 s	**0**	**0**	**no**

Destroying the containers destroyed nothing. `down` removes the containers; the named volumes stay, so the database files stay, so the basket stays. The browser's cookie still pointed at a session that still existed.

`-v` is the whole difference. It deletes the volumes. The shop reinstalled itself from scratch in 17 seconds — which is a good property of the stack and a bad surprise for anyone who typed it to free disk space.

Notice what the third row does *not* say. It does not say the shop broke. It came back, fast, looking perfectly healthy, with every customer's basket gone. Chapter 20 returns to this with backups, where the lesson is the same one Chapter 6 taught about the store identifier: a shop that answers 200 is not thereby a shop that is right.

9.7 The Settings That Govern Any of This

Three of the behaviours above are configurable, and two of the three are in the admin panel rather than in a file.

</> Where each decision is actually made

- **How long an abandoned basket lives.** System › Settings › Edit › Option — the session expiry, `config_session_expire`, 86,400 seconds by default. It governs both the session record and the `DELETE` in the `Cart` constructor, so shortening it discards baskets sooner.
- **Whether a basket needs an account at all.** System › Settings › Edit › Option — **Guest Checkout**. With it on, the basket is keyed by session alone and vanishes with the cookie. With it off, every basket belongs to a customer and follows them to another machine.
- **Where the session itself is kept.** Not in the admin panel at all, but in a file: the `session_engine` line of `system/config/catalog.php`. A setting that changes both a performance characteristic and a deployment constraint, and that can be reached only by editing a file, is worth noticing as a design choice — it is not meant to be changed by the shopkeeper.

Common Pitfalls

- **Assuming where the basket is instead of looking.** Every wrong decision in this chapter follows from that one.
- **Putting a price in the cookie.** Anything the browser carries, the customer can edit. `oc_cart` has a `price` column and the shop recomputes rather than trusting it; your own extensions must do the same.
- **Lengthening session expiry to "keep baskets"** without noticing that it also keeps every robot's session row for the same period.
- **Clearing `oc_session` to tidy up.** It does not delete baskets; it strips them of the identifier that reaches them, leaving rows nothing will ever read and nothing will ever remove.
- **Typing `docker compose down -v` to free space.** It is the one command in this chapter that cannot be undone.
- **Reading a timing from one machine as a property of the software.** The 20% above is a fact about this host.

Try It Yourself: Find the basket, then try to destroy it

```
# --- 1. Predict, in writing, before running anything -----------------
# Where is the basket: cookie, session, or a table? Commit to one.
```

```
# --- 2. Look ------------------------------------------------------
cd code
python m09_basket.py --where

# --- 3. Read the session record yourself --------------------------
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT session_id, LENGTH(data), data FROM oc_session LIMIT 3;"

# --- 4. Find your own basket in the table -------------------------
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT * FROM oc_cart;"

# --- 5. Move the session to files, and check the basket -----------
python m09_basket.py --engine
# Did the basket move? Write down why not, in one sentence.

# --- 6. Count what an empty basket costs --------------------------
python m09_basket.py --cache
# Explain the first row to someone who has not read the source.

# --- 7. Try to destroy it, in increasing order of violence --------
python m09_basket.py --restart
# Predict each row before you read it.

# --- 8. Now the one that works ------------------------------------
# Only when you are ready to reinstall the shop:
python m09_basket.py --restart --destroy

# --- 9. Change the expiry and show that it did something ----------
# Admin: System > Settings > Edit > Option. Set session expiry to 60
# seconds. Add a product, wait, reload. Record what you see and which
# of the two mechanisms in this chapter produced it.
```

What to notice. Step 1 against step 2 is the point of the lab. Keep your written prediction: the gap between what you expected and what the shop does is the only part of this exercise you will still remember in a year.

Checkpoint — Test Yourself

1. A colleague says "move the sessions to Redis and the basket will be faster". Using this chapter's measurements, say what would and would not change, and give the number that settles it.
2. The session table holds 645 rows on a shop that has never had a customer. Explain where they came from, why they are not removed when the visitor leaves, and what you would measure before deciding to shorten the expiry.
3. `docker compose down` left a three-line basket intact and `docker compose down -v` did not. Explain the difference in terms of what each command removes, and name one other thing in your shop that the second command destroys.

Chapter Summary

- HTTP keeps nothing between requests. A cookie is the only standard way to make the second request identifiably related to the first.
- A cookie is a place on the customer's machine; a session is server-side state that *uses* a cookie to point at itself. Confusing the two confuses who can change what.
- Measured: adding three products left the cookie at 49 bytes and the session at 18 bytes, and added three rows to `oc_cart`. The basket in this shop is a table, not a session.

- Measured: 20 first-time visitors created 20 session rows; one visitor making 20 page views created one. Sessions scale with traffic, not with trade.
- Measured: the `db` engine costs exactly two queries per request; the `file` engine costs none and took about 20% off the page on this host — while moving no part of the basket, and while making a second web server impossible.
- Measured: an empty basket is read six times per page and a full one once, because the cache test cannot distinguish "empty" from "not loaded". Every storefront page also deletes from `oc_cart` before reading it.
- Measured: about one request in five rebuilds the session table with `OPTIMIZE TABLE`, at roughly 60 ms.
- Measured: `restart` and `down`/`up` left the basket untouched; `down -v` destroyed it and the shop reinstalled itself in 17 seconds looking perfectly healthy.
- Where state lives decides what survives, what can be reported on, and who is able to tamper with it. It is a design decision, not a detail.

Review Questions

1. **(MCQ)** In the measured shop, adding a product to the basket changes: (a) the cookie; (b) the session record; (c) rows in a table; (d) all three.

2. **(MCQ)** Switching `session_engine` from `db` to `file` moved: (a) the basket; (b) the session record only; (c) both; (d) neither.

3. **(Short)** Explain why a session identifier must be unguessable, and what an attacker who obtained one could do in this shop.

4. **(Short)** Give two things that belong in a cookie and two that must never go in one, with your reason in each case.

5. **(Short)** An empty basket costs six cart queries and a full one costs one. Explain the cause, and propose a one-line change to the condition that would fix it.

6. **(Applied)** A shop reports 400,000 session rows and 12 orders a day. Work out what is producing the rows, state what you would measure to confirm it, and give two changes you would make — one to configuration and one to the shop's robots policy.

7. **(Applied)** You are asked to make baskets survive for thirty days so that customers can come back to them. Say exactly what you would change, what each change costs, what it would break for a guest with no account, and how you would measure whether it increased completed orders at all.